Graphic Era Hill University
DEHRADUN • BHIMTAL • HALDWANI

PROJECT AND TEAM INFORMATION

Project Title

AI-Powered Web-Based C Compiler with Intelligent Debugging and Code Optimization - XYZ

Project GitHub URL

https://github.com/manshi-n/web-based-c-compiler-

Student/Team Information

Team Name:	
Team member 1 (Team Lead) Manshi Rollno- 2319078 Stud id- 230122067 Email- nmansi178@gmail.com Section- DS-1	

Team member 2 Nandini Nautiyal Rollno- 2319150 Stud id- 230121025 Email- nandininautiyal48@gmail.com Section- CS-1	
Team member 3 Tanisha Rollno- 2319723 Stud id- 23012810 Email- tanisha9921@gmail.com Section- G2	
Team member 4 Vedika Gezwal Rollno- 2319843 Stud id- 23012797 Email- vedikagezwal09@gmail.com Section- F2	

PROJECT PROGRESS DESCRIPTION

Project Abstract

(Brief restatement of your project's main goal. Max 300 words).

The AI-Powered Web-Based C Compiler is an intelligent online coding platform designed to compile, execute, debug, and optimize C/C++ programs directly through a browser interface. The project integrates compiler technology with Artificial Intelligence to provide users with automated code analysis, error detection, optimization suggestions, complexity estimation, and security analysis.

The system allows users to write code using the Monaco Editor interface, similar to VS Code, and execute it in real-time using GCC/G++ compilers integrated through a Node.js backend. The platform additionally utilizes the Groq API with Llama models to generate AI-powered debugging suggestions and optimized code solutions.

The project focuses on improving coding efficiency, learning experience, and debugging assistance for students and programmers. Features such as runtime error detection, segmentation fault handling, complexity estimation, safety scoring, syntax highlighting, and AI-based code correction make the platform interactive and educational.

The frontend is designed with a professional modern UI using animations, glassmorphism effects, and responsive layouts to create an industry-level user experience. The backend handles secure code execution, temporary file management, process handling, and API communication.

The project demonstrates practical implementation of compiler construction, AI integration, web development, and system design concepts in a real-world application.

Updated Project Approach and Architecture

(Describe your current approach, including system design, communication protocols, libraries used, etc. Max 300 words).

The project follows a client-server architecture using a web-based frontend and a Node.js backend server. The frontend is developed using HTML, CSS, JavaScript, and Monaco Editor to provide an interactive coding environment. Users can write, edit, and execute C/C++ code directly in the browser.

The backend is built using Express.js and integrates GCC/G++ compilers through child processes for code compilation and execution. Temporary source and executable files are generated dynamically and cleaned automatically after execution. Runtime exceptions such as segmentation faults, infinite loops, and division-by-zero errors are handled using process monitoring and timeout mechanisms.

Artificial Intelligence integration is implemented using the Groq API with the Llama 3.3 model. The backend sends source code and error information to the AI model, which returns optimized code, debugging suggestions, safety analysis, and complexity estimations. Regular expressions are used to extract structured AI responses.

The platform uses REST APIs for communication between frontend and backend modules. Additional system components include safety scoring algorithms, complexity estimation functions, compiler error parsing, and syntax error highlighting. The overall architecture combines web technologies, compiler execution systems, AI-based debugging, and modern UI design to provide an intelligent programming environment.

Tasks Completed

(Describe the main tasks that have been assigned and completed. Max 250 words).

Task Completed	Team Member
Frontend UI development using HTML/CSS/JS	Vedika
Monaco Editor integration	Nandini
Node.js backend development	Tanisha
GCC/G++ compiler integration	Nandini
Runtime execution system	Manshi
AI integration using Groq API	Manshi
Error handling and debugging module	Nandini
Complexity and safety analysis module	Tanisha
UI enhancement and animations	Vedika
GitHub repository setup	Manshi

Challenges/Roadblocks

(Describe the challenges that you have faced. Max 300 words).

Several technical challenges were encountered during project development. One of the major challenges was integrating compiler execution with the Node.js backend while ensuring proper handling of runtime errors, segmentation faults, and infinite loops. Managing temporary files and process cleanup also required careful implementation.

Another significant challenge involved integrating the Groq AI API for intelligent debugging and optimization. The AI responses were sometimes inconsistent or improperly formatted, requiring additional validation logic and extraction mechanisms. Deployment on cloud platforms such as Render introduced further issues related to API connectivity, environment variables, cold starts, and backend timeouts.

Frontend-backend communication initially produced fetch and CORS-related errors, which were resolved through middleware configuration and improved API handling. Designing a professional and responsive user interface while maintaining smooth performance was another challenge.

The project also faced limitations related to free-tier API usage and deployment constraints. Despite these obstacles, most issues were resolved through debugging, optimization, and iterative improvements in system architecture and code structure.

Project Outcome/Deliverables

(Describe what are the key outcomes / deliverables of the project. Max 200 words).

The project successfully delivers an AI-powered web-based compiler capable of compiling, executing, debugging, and analyzing C/C++ programs through a browser interface. Key deliverables include a fully functional compiler execution engine, AI-based debugging and optimization system, Monaco Editor integration, runtime error handling module, complexity estimation system, safety analysis module, and a modern professional user interface.
The system demonstrates successful integration of web technologies, compiler systems, and AI-based code analysis into a unified platform. The project also provides practical exposure to backend development, process management, API integration, and full-stack system design.

Progress Overview

(Summarize how much of the project is done, and what's left. Max 200 words.)

Approximately 85–90% of the project has been completed. Core functionalities such as frontend development, compiler integration, runtime execution, AI-based debugging, complexity estimation, safety analysis, and professional UI implementation are fully operational.
Remaining work primarily involves deployment stabilization, advanced security improvements, cloud optimization, and additional feature implementation such as user authentication, code saving, and multi-language support. The project currently functions successfully in a local development environment and partially on deployment platforms.

Testing and Validation Status

(Provide information about any tests conducted)

Test Type	Status (Pass/Fail)	
Frontend UI Testing	Pass	
Compiler Execution Testing	Pass	
Runtime Error Handling	Pass	
AI Analysis Testing	Pass	
API Communication Testing	Pass	
Deployment Testing	Pass	
Syntax Highlighting Testing	pass	

Deliverables Progress

(Summarize the current status of all key project deliverables mentioned earlier. Indicate whether each deliverable is completed, in progress, or pending.)

The frontend interface, compiler engine, AI integration, debugging system, complexity analysis module, and safety scoring features have been completed successfully. Backend APIs and runtime execution systems are also functional. Deployment configuration and cloud hosting are partially completed but still require optimization for stable AI communication. Future deliverables such as user authentication, cloud storage, and multi-language support are currently in progress.